STOCKVISION

Stock Data Visualization Platform

Techfinity

Team Members:
Anush: Backend Development, Database Integration
Sandesh Nayak: Frontend Development, API Integration
Shreeraksha C H: Frontend Development, API Integration

TechStack Used:
Backend: Spring Boot 3.5.13, Spring Batch, PostgreSQL 17.5
Frontend: Angular 21.2, TypeScript, JWT-Decode, Apex-Charts
Security: JWT Token Authentication, Role-Based Authorization
Authentication: JWT Bearer Tokens

Submission Date: May 8, 2026

1.Solution Overview:

The StockVision is a full-stack web application designed to provide users with a centralized platform for tracking stock market data, visualizing trends, and managing personalized watchlists. The system includes secure authentication, interactive stock charts, and an admin dashboard for controlling stock availability and triggering backend batch synchronization processes.
The application focuses on performance optimization, reduced API dependency, real-time insights, and scalable architecture. By using batch processing and database-driven retrieval, the system ensures efficient stock data management and improved user experience.

Problem:

Managing and analyzing stock market data efficiently becomes challenging when systems rely heavily on external APIs and lack centralized data management. Frequent API calls can lead to request limitations, slower response times, and increased dependency on third-party services.

Additionally, many stock platforms do not provide efficient watchlist management, secure access control, or optimized stock visualization features in a unified system. Users often face difficulties in tracking stock trends, accessing real-time insights, and managing personalized stock data efficiently.

These limitations reduce system performance, affect scalability, and impact overall user experience.

Solution:

StockVision provides a centralized and scalable solution that includes:

- Interactive stock trend visualization using charts
- Personalized watchlist management
- Secure JWT-based authentication and authorization
- Admin dashboard for stock management
- Batch processing for periodic stock data synchronization
- Reduced API dependency through database-driven stock retrieval
- Real-time stock insights with optimized performance
- Scalable full-stack architecture using Angular, Spring Boot, and PostgreSQL

2.User Impact:

- Provides users with quick and secure access to stock market data
- Improves stock analysis through interactive charts and real-time insights
- Enables personalized stock tracking using watchlist management
- Reduces delays caused by repeated external API calls
- Enhances user experience with clean UI, dark/light themes, and secure authentication

Innovation:

- Uses Spring Batch processing to reduce external API dependency
- Implements database-driven stock retrieval for improved performance
- Combines stock visualization, watchlist management, and admin controls in a single platform
- Provides secure JWT-based authentication and role-based access
- Delivers a scalable full-stack architecture with optimized data handling and real-time insights

3. PROBLEM STATEMENT

• Real-world challenge

Stock market platforms often face performance issues due to heavy dependency on external APIs, repeated data fetching, and inefficient stock data management. This leads to slower response times and limited real-time accessibility.

• Who is affected

- Investors and stock market users
- Businesses providing stock analysis services
- Administrators managing stock data and updates

• Evidence/examples

- External stock APIs have request limitations
- Frequent API calls increase response time and dependency
- Existing systems may lack centralized watchlist management and efficient stock visualization features

4. PROPOSED SOLUTION

• Solution description

A full-stack Stock Analysis and Management System that provides stock tracking, interactive chart visualization, personalized watchlists, admin management, and secure authentication.

• How it solves the problem

- Uses batch processing to reduce repeated API calls
- Stores stock data in the database for faster retrieval
- Provides real-time insights through interactive charts
- Ensures secure access using JWT authentication

• Unique value & innovation

- Reduced API dependency using Spring Batch
- Centralized stock management platform
- Real-time stock visualization with personalized watchlists
- Scalable and secure full-stack architecture

5. USE CASES

Use Case 1: User Registration & Login

- **Actors:** User
- **Preconditions:** User must not already have an account
- **Workflow Steps:**
    1. User enters registration details
    2. System validates input fields
    3. User registers successfully
    4. User logs in using credentials
    5. JWT token is generated for secure access
- **Success Criteria:** User is authenticated and redirected to the dashboard

Use Case 2: Stock Data Visualization

- **Actors:** User
- **Preconditions:** User must be logged in
- **Workflow Steps:**
    1. User selects a stock
    2. System fetches stock data from database
    3. Interactive charts display stock trends
- **Success Criteria:** User can view stock performance and trends successfully

Use Case 3: Watchlist Management

- **Actors:** User
- **Preconditions:** User must be authenticated
- **Workflow Steps:**
    1. User adds stocks to watchlist
    2. System stores watchlist data
    3. User can view or remove stocks anytime
- **Success Criteria:** Personalized watchlist is managed successfully

Use Case 4: Admin Stock Synchronization

- **Actors:** Admin
- **Preconditions:** Admin must be logged in
- **Workflow Steps:**
    1. Admin triggers batch synchronization
    2. System fetches stock data from external APIs
    3. Updated data is stored in database
- **Success Criteria:** Stock data is updated successfully with reduced API dependency

6. ARCHITECTURE & DESIGN

Architecture Diagram

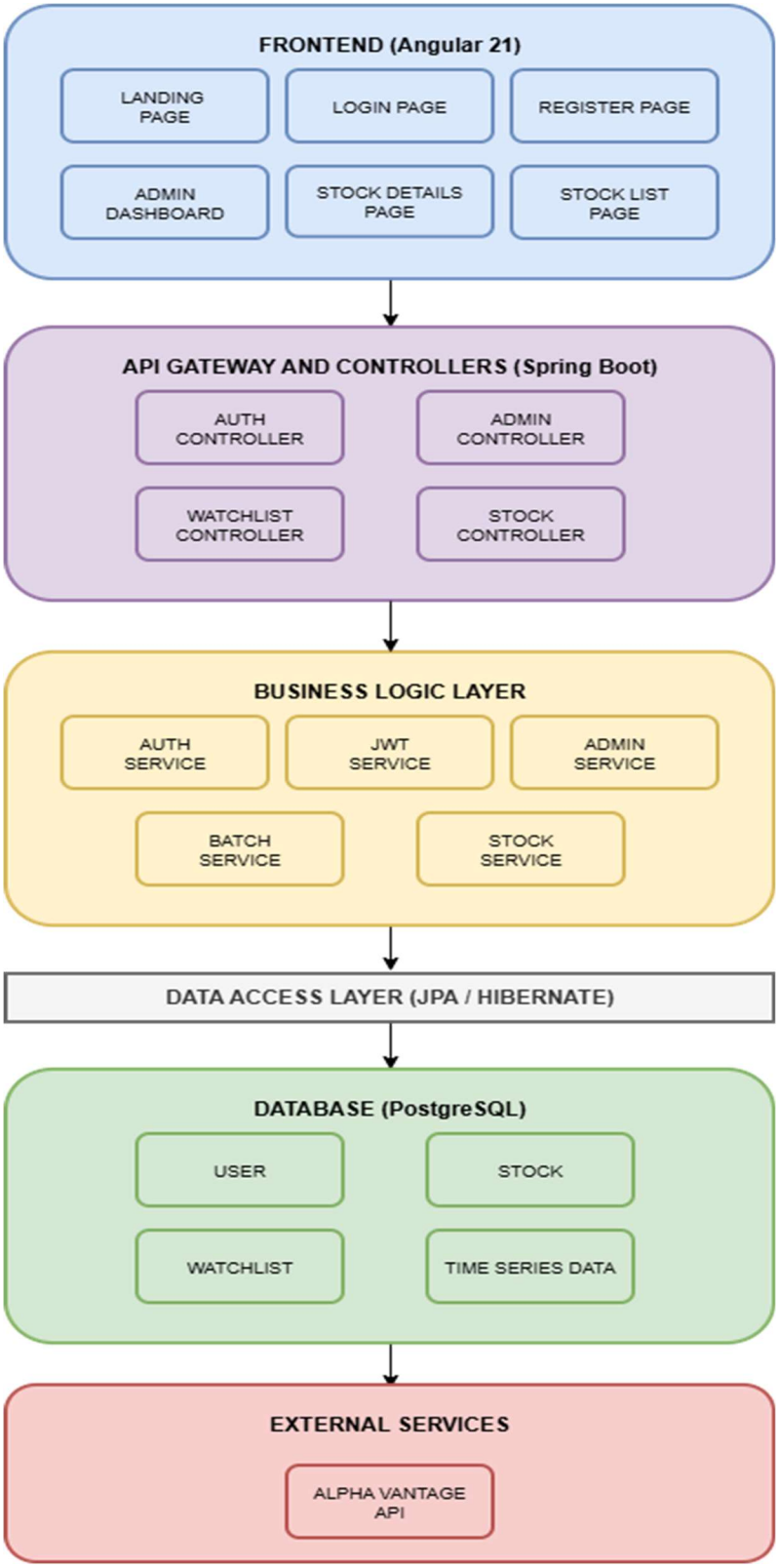

Fig: Architecture Diagram

Workflow/Sequence Diagram: Customer Places Order

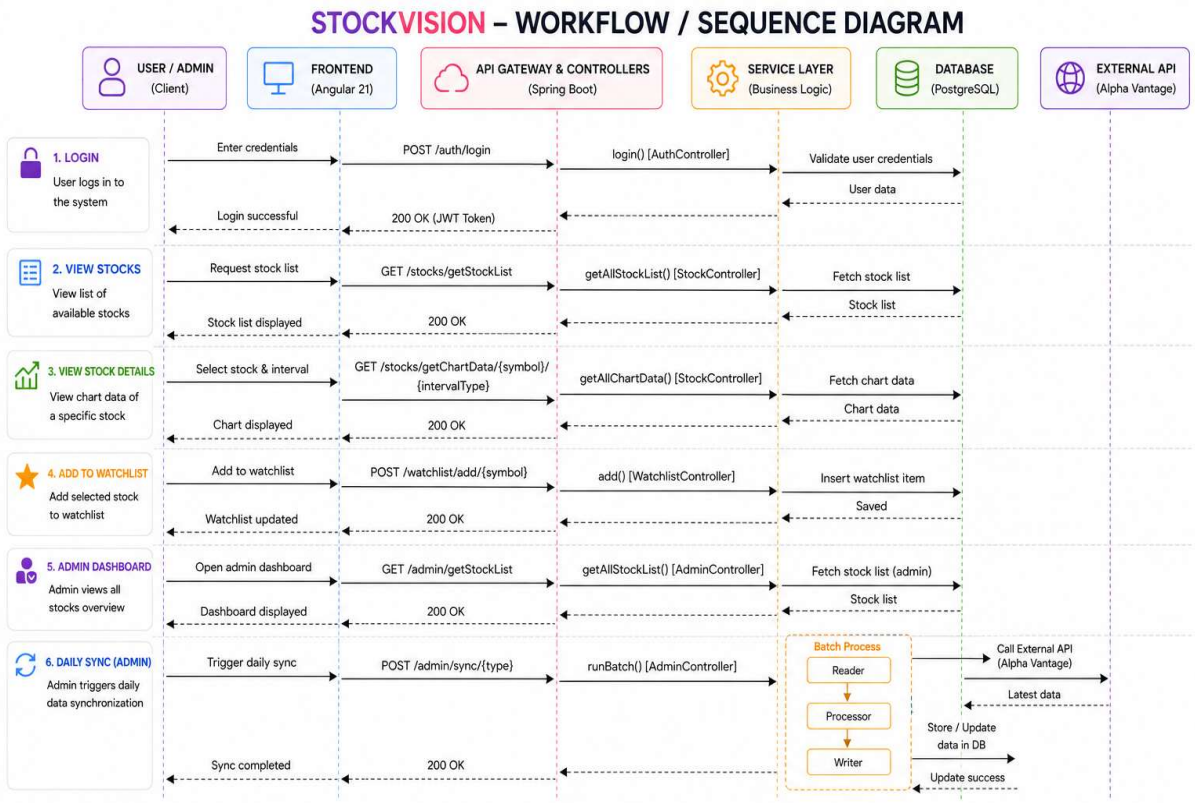

Fig: Workflow/Sequence Diagram

Technology Stack Details:

Component	Technology	Version	Purpose
Frontend Framework	Angular	19+	UI development and routing
Frontend Language	TypeScript	5+	Type-safe frontend development
Styling	CSS3	Latest	Responsive UI design
HTTP Client	Angular HttpClient	19+	Backend API communication
Backend Framework	Spring Boot	3+	REST API development
Backend Language	Java	17	Backend business logic
ORM Framework	Spring Data JPA / Hibernate	6+	Database interaction and mapping

Database	PostgreSQL	15+	Relational database management
Authentication	JWT & Spring Security	N/A	Secure authentication and authorization
Batch Processing	Spring Batch	5+	Stock data synchronization
Charts & Visualization	ApexCharts	Latest	Interactive stock charts
Build Tool	Maven	Latest	Dependency and project management
API Testing	Postman	Latest	REST API testing
Version Control	Git & GitHub	Latest	Source code management

7. FEATURES & CAPABILITIES

Admin features

Feature	Description	Business Value	Technical Value
Stock Management	Add and manage stock details	Simplifies stock administration	CRUD operations with REST APIs
Batch Synchronization	Trigger stock data updates	Reduces API dependency	Spring Batch integration
Dashboard Monitoring	View stock and system insights	Better monitoring and analysis	Dynamic dashboard integration

User features

Feature	Description	Business Value	Technical Value
View Stocks	View available stock data	Improves stock visibility	Real-time database retrieval
Stock Visualization	Analyze trends using charts	Better market insights	Interactive ApexCharts integration
Watchlist Management	Add/remove personalized stocks	Personalized user experience	Database-based watchlist handling
Secure Login	Access system securely	Improves data protection	JWT authentication
Real-Time Updates	View updated stock information	Better user engagement	Optimized API and DB synchronization
Dashboard Access	View market summaries and trends	Faster decision-making	Dynamic Angular UI updates

8. INSTALLATION GUIDE

System Requirements:

1. OS: Windows 10/11
2. Backend: Java 17+, Spring Boot, Spring Batch
3. Database: PostgreSQL 13+
4. Frontend: Node.js 18+, npm 10+, Angular CLI
5. RAM: 8GB minimum
6. Storage: 2GB for dependencies
7. Ports: 8080 (Backend), 4200 (Frontend)

Backend Setup (Spring Boot):

1. **Install Prerequisites:**

   - Install **Java 17+**
   - Install **Maven** (or use Spring Boot embedded Maven)

   Verify installation

   command: java -version

   mvn -version

2. **Navigate to Backend Folder:**

   Command: cd StockBackend

3. **Configure the database**

```
# ==========================
# POSTGRES DB CONFIG
# ==========================
spring.datasource.url=jdbc:postgresql://localhost:5432/stockdb
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

4. **Create PostgreSQL Database**

   command: CREATE DATABASE stockdb;

5. **Build Project**

   command: mvn clean install

6. **Run Backend:**

   command: mvn spring-boot:run

Frontend Setup (Angular)

1. **Install Node.js:**
   Download from https://nodejs.org
   Verify: **node --version** and **npm –version**
2. **Install Angular CLI:**
   Command: npm install -g @angular/cli
3. **Navigate to Frontend Folder:**
   Command: **cd StockFrontend**
4. **Install Dependencies:**
   Command: **npm install**
5. **Configure Backend URL:**
   Update API URLs in service files:
   private apiUrl= 'http://localhost:8080';
   private adminUrl = 'http://localhost:8080/admin';
   private authUrl = 'http://localhost:8080/auth';

6. **Run Frontend Development Server:**
   Command: **ng serve**
   Application available at: http://localhost:4200
   Auto-reloads on file changes

Database Setup

1. **Database Name:** stockdb
2. **Tables Created:**
   → Users- users(id, password, role, username)
   → Stock – stock(id, description, cmp_name, symbol, daily_updated_date, is_active, monthly_updated_date, weekly_updated_date)
   → TimeSeriesData – time_series_data(id, close, date, hight, low, open, stock_id, volume, interval_type)
   → Watchlist – watchlist(id, stock_id, user_id)
3. **Schema Handling**
   Managed using Spring Data JPA (Hibernate)
   Tables automatically created using : spring.jpa.hibernate.ddl-auto=update

   Environment Variables Configuration
   Backend (application.properties):
   spring.application.name=StockBackend
   server.port=8080

   # ==========
   # ALPHA VANTAGE API CONFIG

```
# ==========================
ALPHAVANTAGE_BASEURL=https://www.alphavantage.co/query
ALPHAVANTAGE_APIKEY= <ALPHA_VANTAGE _KEY>

# ==========================
# SPRING BATCH CONFIGURATION
# ==========================
spring.batch.jdbc.initialize-schema=always
spring.batch.job.enabled=false

# ==========================
# LOGGING
# ==========================
logging.level.org.springframework.batch=INFO

# ===========================
# JWT CONFIG
# ===========================
jwt.secret=<JWT_SECRET _KEY>
jwt.expiration=3600000

spring.security.user.name=none
spring.security.user.password=none
```

9. USER MANUAL

Login Steps

1. Open the application in browser
2. Navigate to Login Page
3. Enter registered Username and Password
4. Click Login
5. JWT token is generated for secure access
6. User is redirected to the Stocks Dashboard

Input Steps

User Registration

1. Enter Username and Password
2. System validates input fields
3. Click Register after validations are satisfied
4. User redirected to Login Page

Stock Search & Watchlist

1. Login to the application
2. Search or select stock
3. View stock charts and market data
4. Add or remove stocks from watchlist

Admin Operations

1. Login as Admin
2. Trigger stock synchronization process
3. Manage stock availability and updates

Triggering Workflow

User Workflow

1. User registers and logs in
2. JWT authentication validates access
3. User accesses stock dashboard
4. System retrieves stock data from database
5. User views charts and manages watchlist

Admin Workflow

1. Admin logs into dashboard
2. Admin triggers batch synchronization
3. System fetches stock data from external APIs
4. Updated stock data stored in database

Viewing Outputs

Stocks Dashboard

Displays:

- Stock market data
- Interactive charts
- Price trends and updates

Watchlist Dashboard

Displays:

- Personalized saved stocks
- Stock performance tracking

Admin Dashboard

Displays:

- Stock synchronization status
- Managed stock details
- Batch update operations

Troubleshooting:

Issue	Cause	Solution
Login Failed	Incorrect credentials	Verify username and password
Unauthorized Access	Invalid/Expired JWT token	Logout and login again
Unable to Fetch Stock Data	Backend or API issue	Restart backend service
Charts Not Loading	Network/API delay	Refresh the page
Watchlist Not Updating	Database sync issue	Reload dashboard
Database Connection Failed	PostgreSQL not running	Start PostgreSQL service

10. DEMO INSTRUCTIONS

Demo Video Link:

[StockVision Demo.mp4]

Sample Data:

Pre-populated Database:

Users:

- ID: 1, Username:Admin, Password: <hashed_password>, Role: ADMIN

11. INNOVATION & IMPACT

Business Impact

- Reduces dependency on external stock APIs
- Improves stock data management and performance
- Provides centralized stock tracking and administration
- Enhances decision-making with real-time insights

User Impact

- Enables easy stock tracking and watchlist management
- Provides interactive charts for better market analysis
- Improves user experience with clean UI and theme switching
- Ensures secure access using JWT authentication

Technical Innovation

- Uses Spring Batch for automated stock synchronization
- Database-driven stock retrieval reduces repeated API calls
- Real-time visualization using interactive chart integration
- Secure JWT-based authentication and authorization

Scalability Benefits

- Scalable full-stack architecture using Angular and Spring Boot
- Optimized database handling with PostgreSQL
- Supports efficient large-scale stock data management
- Modular backend design for future feature expansion

12. TECHNICAL DETAILS

API Specifications

Base URL: http://localhost:8080/

Authentication: All endpoints except /auth/register and /auth/login require JWT token in header:

Authorization: Bearer <jwt_token>

Auth Endpoints:

POST /auth/register

Request: { username, password}

Response: { message: "User registered successfully", username, role }

POST /auth/login

Request: { username, password }

Response: {token, username, role }

Admin Endpoints:

GET /admin/getStockList

Response: [{ symbol, cmpName, isActive, dailyUpdatedDate, weeklyUpdatedDate, monthlyUpdatedDate }, …]

Authorization: Admin

PUT /admin/toggle/{symbol}

Response: {id, symbol,description, cmpName, isActive, dailyUpdatedDate, weeklyUpdatedDate, monthlyUpdatedDate }

Authorization: Admin

POST /admin/sync/{type}

Request: {type}

Response: {updated, skipped, apiLimitReached, message}

Authorization: Admin

Stock Endpoints:

GET /stocks/getStockList

Response: { symbol, cmpName, closePrice, price, percentage, status}

Authorization: User

GET /stocks/getChartData/{symbol}/{intervalType}

Request: {symbol, intervalType}

Response: [{id, {id, symbol,description, cmpName, isActive, dailyUpdatedDate, weeklyUpdatedDate, monthlyUpdatedDate } , date, open, high, low, close, volume, intervalType}, …]
Authorization: User

Watchlist Endpoints:

GET /watchlist/get
Response: { symbol, cmpName, closePrice, price, percentage, status}
Authorization: User

POST /watchlist/add/{symbol}
Request: {symbol}
Response: {message: "Stock added to watchlist"}
Authorization: User

DELETE /watchlist/delete/{symbol}
Request: {symbol}
Response: {message: "Stock removed from watchlist"}
Authorization: User

Data Models

User Model:

```
public class User {
   private Long id;
   private String username;
   private String password;
   private Role role;
}
```

Stock Model:

```
public class Stock {
   private Long id;
   private String symbol;
   private String cmpName;
   private String description;
   private boolean isActive = true;
   private LocalDate dailyUpdatedDate;
   private LocalDate weeklyUpdatedDate;
   private LocalDate monthlyUpdatedDate;
}
```

TimeSeriesData Model:

```
public class TimeSeriesData {
   private Long id;
   private Stock stock;
   private LocalDate date;
   private BigDecimal open;
   private BigDecimal high;
   private BigDecimal low;
   private BigDecimal close;
   private Long volume;
   private String intervalType;
}
```

Watchlist Model:

```
public class Watchlist {
   private Long id;
   private User user;
   private Stock stock;
}
```

Security

1. **Authentication:**

   JWT Bearer Token authentication with secure login system
   Token generated after successful login
   Token contains:
   - Username
   - User Role
   - IssuedTime
   - ExpiryTime

   JWT token stored in browser localStorage
   Role-based redirection after login

2. **Authorization:**

   Role-Based Access Control (RBAC) implemented

   Separate access for:

   - Admin
   - User

   Admins can access only Admin Dashboard.
   User can access stock list page and stock details page.

3. **Data Protection:**

   Passwords stored securely in encrypted form
   Role-based data isolation improves security
   Secure API communication between frontend and backend
   Input validation implemented for all user operations
   Prevents unauthorized access

4. **API Security:**

   JWT token verification for protected endpoints
   Exception handling prevents sensitive data exposure
   Backend prevents unauthorized resource access

Performance

1. Database Optimization:

- PostgreSQL used for efficient stock data management
- Spring Data JPA/Hibernate optimizes database operations
- Structured entity relationships improve data consistency and retrieval

2. API Performance:

- REST APIs enable fast frontend-backend communication
- Batch processing reduces repeated external API calls
- Optimized stock retrieval improves response time

3. Frontend Performance:

- Angular standalone components improve modularity
- Dynamic UI updates without full page reload
- Optimized dashboard and chart rendering enhance user experience

4. Real-Time Updates:

- Stock data refreshed through scheduled batch synchronization
- Watchlist and stock dashboards update dynamically
- Interactive charts reflect updated stock trends efficiently

Scalability

1. Database Scalability:

- PostgreSQL supports large-scale stock and watchlist data management
- Structured database design supports future expansion for more users and stock datasets

2. Application Scalability:

- Separate Angular frontend and Spring Boot backend improve maintainability
- REST API architecture supports future microservices-based expansion

3. Future Expansion:

The system can be extended with:

- Real-time stock market API integration
- Mobile application support
- Notification and alert services
- Advanced analytics dashboards and AI-based stock predictions

13. LIMITATIONS

1. Real-Time Data

- Stock data updates depend on batch synchronization
- Continuous live market streaming is not fully implemented

2. Dashboard Features

- Limited advanced analytics and reporting features
- AI-based prediction and recommendation system not included

3. Notifications

- No email or push notification system available
- Users are not automatically alerted for stock updates

4. Mobile Support

- Application mainly optimized for desktop usage
- Mobile responsiveness can be improved further

5. External Integration

- Limited integration with external financial platforms
- System currently focused on stock tracking and visualization

Mock Data

- Sample stock data and watchlists were created for testing
- Dummy user accounts used for authentication testing
- Mock stock trends used for chart visualization and dashboard features

Time Constraints

- Advanced real-time analytics not fully implemented
- No export options for reports or watchlists
- Cloud deployment and microservices architecture not completed
- Advanced filtering and AI-based insights limited in current version

14. FUTURE ENHANCEMENTS

1. User Experience

- Live stock market integration with real-time updates
- Mobile application support
- Notifications for stock alerts and watchlist updates
- Export options for stock reports and watchlists

2. Admin Features

- Advanced stock analytics dashboard
- Automated stock synchronization scheduling
- Detailed system monitoring and reporting

3. Analytics Enhancements

- AI-based stock prediction and recommendations
- Interactive advanced charts and comparison reports
- Portfolio performance analytics

4. Security Enhancements

- Two-Factor Authentication (2FA)
- Password recovery and email verification

Integrations

1. Third-party Services

- Real-time stock market API integration
- Cloud deployment using AWS/Azure
- Notification and email services

2. Analytics

- Advanced stock trend analysis
- Real-time market insights and forecasting

Production Readiness

1. DevOps

- Docker containerization
- CI/CD pipeline integration
- Monitoring and logging support

2. Scalability

- Microservices-ready architecture
- Database optimization for large datasets
- Redis caching for faster stock retrieval

3. Security

- OAuth2 authentication support
- Multi-factor authentication
- Security testing and audit logs

4. Performance

- Optimized REST APIs
- Faster dashboard rendering
- Improved batch processing efficiency

15. CONCLUSION

The StockVision provides a secure and centralized platform for stock tracking, visualization, and watchlist management. By implementing JWT authentication, batch processing, and interactive dashboards, the system ensures efficient and optimized stock data handling.

The project successfully delivers:

- Secure user authentication and authorization
- Interactive stock visualization and trend tracking
- Personalized watchlist management
- Reduced API dependency using batch processing
- Scalable full-stack architecture for future enhancements

The system effectively demonstrates real-world implementation of full-stack development, secure authentication, database management, and scalable stock data processing.

16. APPENDIX

API Reference

Complete Endpoint List:

POST /auth/register
POST /auth/login

GET /admin/getStockList
PUT /admin/toggle/{symbol}
POST /admin/sync/{type}

GET /stocks/getStockList
GET /stocks/getChartData/{symbol}/{intervalType}

GET /watchlist/get
POST /watchlist/add/{symbol}
DELETE /watchlist/remove/{symbol}

SQL Scripts

Database Creation:

```
CREATE DATABASE stockdb;
USE stockdb;
```

1. **User Table**

```
CREATE TABLE users (
  Id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(255) NOT NULL UNIQUE,
  role VARCHAR(255) NOT NULL,
  password VARCHAR(255) NOT NULL
);
```

2. **Stock Table**

```
CREATE TABLE stock (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  description VARCHAR(5000) NOT NULL,
  cmp_name VARCHAR(255) NOT NULL,
  symbol VARCHAR(255) NOT NULL UNIQUE,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  daily_updated_date DATE,
  weekly_updated_date DATE,
  monthly_updated_date DATE
);
```

3. **TimeSeriesData Table**

```
CREATE TABLE time_series_data(
  Id BIGINT PRIMARY KEY AUTO_INCREMENT,
 stock_id BIGINT NOT NULL,
 date DATE,
 open DECIMAL(19,4),
 high DECIMAL(19,4),
 low DECIMAL(19,4),
 close DECIMAL(19,4),
 volume BIGINT,
 interval_type VARCHAR(50),
 CONSTRAINT fk_time_series_stock
     FOREIGN KEY (stock_id)
     REFERENCES stock(id)
     ON DELETE CASCADE,
 CONSTRAINT uk_time_series_stock_date_interval
     UNIQUE (stock_id, date, interval_type)
);
```

4. **Watchlist Table**

```
CREATE TABLE watchlist (
   id BIGINT PRIMARY KEY AUTO_INCREMENT,
   user_id BIGINT NOT NULL,
   stock_id BIGINT NOT NULL,
   CONSTRAINT fk_watchlist_user
      FOREIGN KEY (user_id)
      REFERENCES users(id)
      ON DELETE CASCADE,
   CONSTRAINT fk_watchlist_stock
      FOREIGN KEY (stock_id)
      REFERENCES stock(id)
      ON DELETE CASCADE,
   CONSTRAINT uk_watchlist_user_stock
      UNIQUE (user_id, stock_id)
);
```